

1. What is XSS?

Cross-Site Scripting (XSS) is a **web security vulnerability** that allows an attacker to **inject malicious scripts** into a website, which are then executed in another user's browser.

Key idea:

The website **fails to properly validate or escape user input**, so attackers can inject **JavaScript (usually)** that runs in someone else's browser.

Example scenario:

- A comment box on a blog lets users post text.
- Attacker posts:

```
<script>alert("Hacked!")</script>
```

- If the website displays this comment without filtering, **every visitor** who views the page will see an alert box.

2. How XSS Works

1. **Attacker finds input field** or URL parameter that is not sanitized.
2. **Attacker injects malicious script** (mostly JavaScript).
3. **Server or client-side script processes it without proper filtering.**
4. **Script executes in victim's browser** as if it came from the trusted site.
5. Attacker can:
 - Steal cookies/session tokens.
 - Redirect to malicious sites.
 - Deface the site for that user.
 - Log keystrokes or capture credentials.

3. Types of XSS

(A) Stored XSS (Persistent)

- **Malicious script is stored** on the server (DB, message board, profile, etc.).
- Every time a user loads that page, the script runs.
- **Danger level:** 🔥 High (affects multiple users).
- **Example:** A profile bio containing <script>...</script>.

(B) Reflected XSS (Non-Persistent)

- Script is **immediately reflected** in the response (usually via URL parameters).
- Requires the victim to click a **crafted malicious link**.
- **Danger level:** Medium (needs social engineering).
- **Example:**

[https://example.com/search?q=<script>alert\(1\)</script>](https://example.com/search?q=<script>alert(1)</script>)

(C) DOM-based XSS

- Happens **entirely in the browser**, without server modification.
- JavaScript on the page reads data from URL/hash and **inserts it into the DOM** without sanitizing.
- **Danger level:** Medium-High (depends on JS code).
- **Example:**

```
document.getElementById("result").innerHTML = location.hash;
```

Visiting `example.com/#<script>alert(1)</script>` will execute the script.

4. Precaution / Prevention Methods

(A) Input Validation

- Allow only expected characters (whitelist approach).
- For example: Names → only letters, spaces.
- Reject <, >, &, ", ', /.

(B) Output Encoding

- Encode data before displaying in HTML, JS, or attributes.
- Use libraries like:
 - PHP → htmlspecialchars()

- JavaScript → DOM methods instead of .innerHTML

(C) Use Content Security Policy (CSP)

- Add HTTP header to block inline scripts and only allow scripts from trusted sources.
- Example:

Content-Security-Policy: script-src 'self'

(D) Escape Special Characters

- Convert < to <, > to >, " to ", ' to '.

(E) Avoid Dangerous Functions

- Avoid eval(), innerHTML, document.write() with user input.

(F) Use Security Framework Features

- Frameworks like **React, Angular, Django** automatically escape output.

(G) Regular Security Testing

- Use tools like:
 - **OWASP ZAP**
 - **Burp Suite**
 - Browser dev tools
- Test inputs and URLs.